only used to backpropagate and train the inverse and encoder models. First we'll look at the hyperparameter setup and the instantiation of the models.

Listing 8.8. Hyperparameters and model instantiation

```
params = {
    'batch_size':150,
    'beta':0.2,
    'lambda':0.1,
    'eta': 1.0,
    'gamma':0.2,
    'max_episode_len':100,
    'min_progress':15,
    'action_repeats':6,
    'frames_per_state':3
}
replay = ExperienceReplay(N=1000, batch_size=params['batch_size'])
Qmodel = Qnetwork()
encoder = Phi()
forward_model = Fnet()
inverse_model = Gnet()
forward_loss = nn.MSELoss(reduction='none')
inverse_loss = nn.CrossEntropyLoss(reduction='none')
qloss = nn.MSELoss()
all_model_params = list(Qmodel.parameters()) +
list(encoder.parameters())  1
all_model_params += list(forward_model.parameters()) +
list(inverse_model.parameters())
opt = optim.Adam(lr=0.001, params=all_model_params)
```

- ***1*** We can add the parameters from each model into a single list and pass that into a single optimizer.

Some of the parameters in the `params` dictionary will look familiar, such as `batch _size`, but the others probably don't. We'll go over them, but first let's take a look at the overall loss function.

Here's the formula for the overall loss for all four models (including the DQN):

$$minimize[\lambda \times Q_{loss} + (1 - \beta)F_{loss} + \beta \times G_{loss}]$$

This formula adds the DQN loss to the forward and inverse model losses, each scaled by a coefficient. The DQN loss has a free-scaling parameter, λ,